Software Requirements Specification (SRS): Top5ive

Version: 2.0

Date: January 30, 2026

Developer: Daysha Roberts

Repository: https://github.com/xTop5ive/top5ive

1.1 Purpose

The purpose of this document is to specify the requirements for **Top5ive**, a social music curation platform. Top5ive addresses the complexity of modern streaming interfaces by providing a streamlined, community-centric environment for playlist organization and discovery. This SRS serves as the definitive technical agreement for development through the April 2026 milestone.

1.2 Scope

Top5ive is a web-based application that allows users to curate personal music collections, engage with a social community through likes and follows, and perform collaborative curation. The system focuses on metadata management and rhythmic discovery via Beats Per Minute (BPM) rather than direct audio hosting.

1.3 Definitions/Glossary

- **BPM (Beats Per Minute):** A numerical measure of tempo used to determine rhythmic similarity.
- **Collaborator:** A secondary user role with permissions to modify a shared playlist owned by another user.
- **Magic Link:** An authentication method where a user receives a unique URL via email to sign in without a password.
- **Metadata:** Descriptive data about a track, including artist name, duration, and BPM.

1.4 References

1. **Project Repository:** https://github.com/xTop5ive/top5ive.
2. **Next.js Documentation:** Official framework documentation for App Router patterns.
3. **Supabase Documentation:** Technical guides for Auth, PostgreSQL, and Storage services.

---

2. Overall Description

2.1 Product Perspective & Context

Top5ive operates as a standalone web application utilizing a modern serverless architecture. The system interfaces with the **Spotify Web API** and **YouTube Music** to retrieve music metadata, ensuring a robust library of song information for user playlists.

2.2 User Classes

- **Standard User:** An authenticated individual capable of creating personal playlists, managing their profile, and interacting with the community.
- **Collaborator:** An authorized user who has been granted editing rights to a specific playlist by its owner.
- **Admin (Future):** A privileged role intended for system moderation and community guideline enforcement.

2.3 Operating Environment

The application is built using **Next.js 16 (App Router)** for the frontend and API layers. Data persistence is managed via **Prisma ORM** connected to a **PostgreSQL** database hosted on **Supabase**. Authentication is handled exclusively through **Supabase Auth**.

2.4 Constraints & Assumptions

- **API Constraints:** System performance is dependent on the uptime and rate limits of external music metadata providers.
- **Storage Limits:** Individual playlist cover uploads are limited to 2 MB per file.
- **Assumption:** Users are expected to have a stable internet connection and a modern web browser supporting HTML5 and JavaScript.

---

3. System Features (Functional Requirements)

ID	Feature Name	Acceptance Criteria (Given/When/Then)
FR-001	**Magic Link Auth**	**Given** a user enters their email, **When** they click "Sign In," **Then** a secure magic link is sent to their inbox.
FR-002	**Playlist Creation**	**Given** a title and visibility setting, **When** a user submits the form, **Then** a new playlist record is saved to the database.
FR-003	**Tag Parsing**	**Given** a comma-separated string, **When** a user saves a playlist, **Then** individual tag records are created and linked to the playlist.

ID	Feature Name	Acceptance Criteria (Given/When/Then)
FR-004	**Track Model**	**Given** valid metadata (BPM, Artist, URL), **When** a user adds a track, **Then** it is appended to the playlist's track list at the correct position.
FR-005	**Social Interaction**	**Given** a public playlist, **When** a user clicks "Like," **Then** the like count increments and the playlist appears in the user's "Liked" tab.
FR-006	**Cover Upload**	**Given** an image under 2 MB, **When** uploaded to Supabase Storage, **Then** the playlist's cover_url is updated with the public file link.
FR-007	**Collaboration**	**Given** a valid collaborator handle, **When** the owner adds them, **Then** the collaborator gains song editing permissions for that specific playlist.

---

4. External Interface Requirements

4.1 User Interfaces

The user interface shall utilize a responsive grid layout that adapts to desktop and mobile viewports. Key components include a navigation sidebar, a centralized search bar for music discovery, and consistent song cards displaying BPM and duration.

4.2 API Endpoints

- POST /api/playlists: Create a new playlist.
- GET /api/explore: Retrieve playlists with filtering for tags and sorting by newest/most liked.
- PATCH /api/tracks/add: Append a new track to a specified playlist.
- DELETE /api/playlists/[id]: Restricted endpoint for owners to remove content.

---

5. Non-Functional Requirements

- **Performance (NFR-001):** The Explore page shall return filtered results within 1 second of user input under normal network conditions.

- **Security (NFR-002):** All database access must be authorized through Row Level Security (RLS) policies within Supabase.
- **Availability (NFR-003):** The system shall maintain 99% uptime for core playlist viewing and searching features.
- **Usability (NFR-004):** New users shall be able to create their first playlist within 3 minutes of their initial magic link login.

---

6. Data Model & Architecture

6.1 Entity-Relationship Diagram (ERD)

Code snippet

```
erDiagram
  PROFILE ||--o{ PLAYLIST : "owns"
  PROFILE ||--o{ LIKE : "gives"
  PROFILE ||--o{ COLLABORATION : "is invited to"
  PLAYLIST ||--o{ LIKE : "receives"
  PLAYLIST ||--o{ PLAYLIST_TAG : "has"
  PLAYLIST ||--o{ TRACK : "contains"
  PLAYLIST ||--o{ COLLABORATION : "allows"
  TAG ||--o{ PLAYLIST_TAG : "categorizes"

  PROFILE {
    string id PK
    string username
    string handle
  }
  PLAYLIST {
    string id PK
    string title
    boolean isPublic
    string cover_url
  }
  TRACK {
    string id PK
    string title
    string artist
    int bpm
    int position
  }
  TAG {
    string id PK
    string name
  }
```

---

7. Use Cases

7.1 Use Case Diagram

7.2 Primary Use Case: Add Track (New)

- **Primary Actor:** User or Collaborator.
- **Precondition:** User is authenticated and has editing permissions for the selected playlist.
- **Flow:**
    1. User selects "Add Track" on a playlist detail page.
    2. User inputs track metadata or selects from search results.
    3. System validates the input and assigns the next available position index.
    4. System saves the track and updates the UI for all viewers.

---

8. Traceability Matrix

Goal	Use Case	Functional Req	Non-Functional Req
Social Growth	Like Playlist	FR-005	NFR-001 (Performance)
Curation	Add Track	FR-004	NFR-003 (Availability)
Privacy	Create Private Playlist	FR-002	NFR-002 (Security)
Community	Add Collaborator	FR-007	NFR-004 (Usability)

---

9. Risks & Mitigations

- **Risk:** External API downtime prevents track addition.
    - **Mitigation:** Implement client-side caching of common search results to allow partial offline functionality.
- **Risk:** Unauthorized deletion of playlists via API exploits.

- **Mitigation:** Enforce strict Server-Side ownership validation in Next.js API routes using Prisma and Supabase IDs.

---

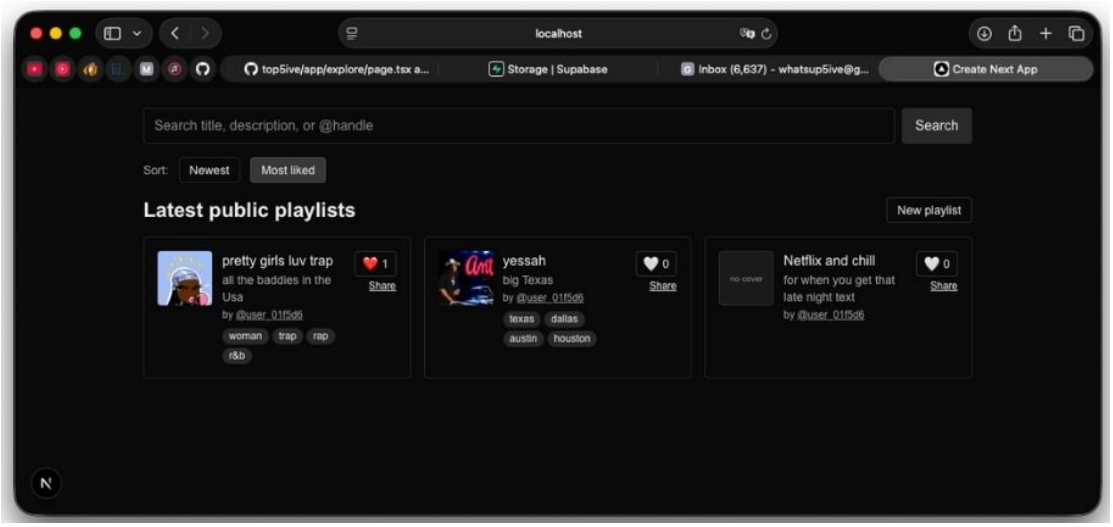
10. Project Plan & Gantt (Spring 2026)

- **Jan 30 - Feb 15:** Implement Track Model in Prisma and create POST /api/tracks endpoint.
- **Feb 16 - Mar 5:** Develop "Add Track" UI and search integration.
- **Mar 6 - Mar 25:** Integrate Supabase Storage for cover image uploads.
- **Mar 26 - Apr 10:** Create Dashboard "Liked Playlists" tab and empty state UI polish.
- **Apr 11 - Apr 27:** Final QA, bug fixes, and production deployment.

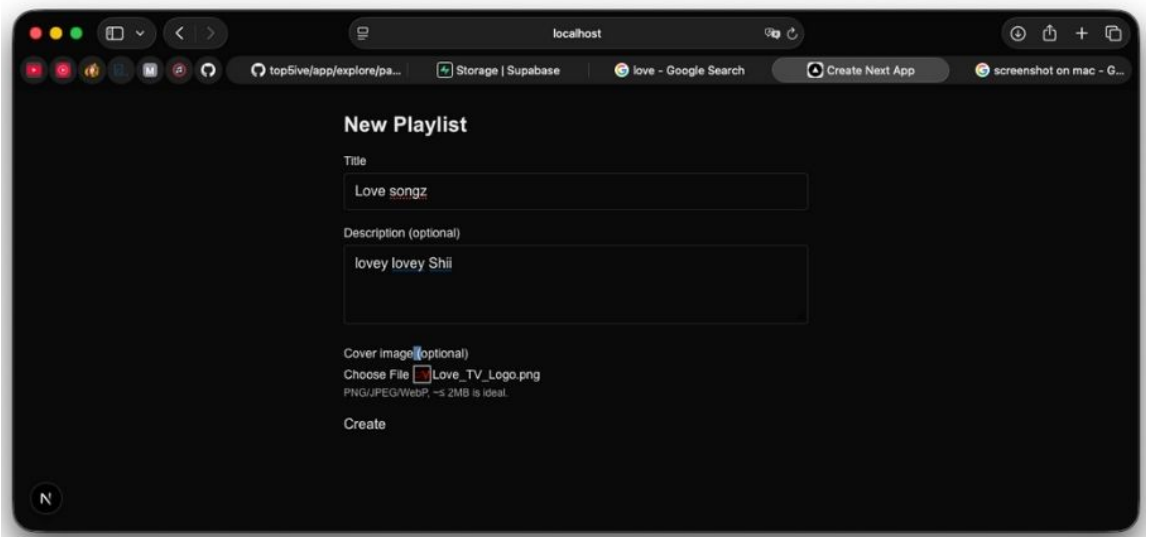
---

11. Appendices

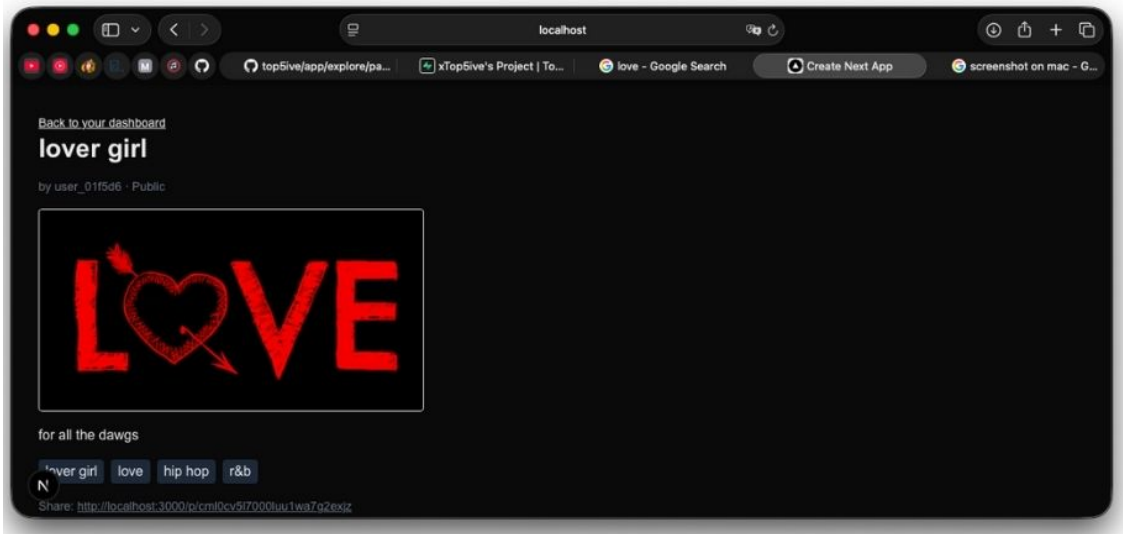
Appendix B: UI Screenshots



- **[Screenshot: Explore with tags and sort]**



- **[Screenshot: New Playlist form]**



- **[Screenshot: Playlist detail with tags]**

Change Log (Summary of Improvements)

1. **Link Correction:** Fixed the GitHub link to the correct repository: https://github.com/xTop5ive/top5ive.
2. **Writing Quality:** Converted all fragmented notes into formal, complete sentences.
3. **Collaborator Actor:** Introduced the "Collaborator" role across the SRS, ERD, and Use Case sections.
4. **Acceptance Criteria:** Added specific Given/When/Then scenarios for every functional requirement.
5. **Project Plan:** Created a new Gantt-aligned schedule covering milestones through April 27, 2026.